

1. Non-Recursive CTE (Common Table Expression)

A temporary result set that exists only for the duration of a query. It improves readability and simplifies complex queries.

```
WITH high_salary AS (  
  SELECT  
    u.user_name,  
    s.salary_amount  
  FROM users u  
  JOIN salary s  
    ON u.user_id = s.user_id  
  WHERE s.salary_amount > 70000  
)  
SELECT *  
FROM high_salary;
```

Use Case: Breaking complex queries into smaller logical steps.

2. Recursive CTE

A CTE that references itself. Commonly used for hierarchical or tree-structured data.

```
WITH RECURSIVE numbers AS (  
  SELECT 1 AS n  
  UNION ALL
```

```
SELECT n + 1
FROM numbers
WHERE n < 10
)
SELECT *
FROM numbers;
```

Use Case: Organizational hierarchies, category trees, bill of materials.

```
SELECT
  u.user_name,
  s.salary_amount,
  CASE
    WHEN s.salary_amount >= 90000 THEN 'High'
    WHEN s.salary_amount >= 60000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_category
FROM users u
JOIN salary s
  ON u.user_id = s.user_id;
```

Yes. If you're teaching the concepts, you can demonstrate the same logic using only `SELECT` queries without creating functions.

3. Scalar Function Example

A scalar expression returns one value per row.

```
SELECT
  user_id,
  salary_amount,
  salary_amount * 12 AS annual_salary
FROM salary;
```

Purpose: Returns a single value (annual salary) for each row.

4. Aggregate Function Example

Returns one summarized value from multiple rows.

```
SELECT
  SUM(salary_amount) AS total_salary
FROM salary;
```

Other aggregate functions:

```
SELECT
    COUNT(*) AS employee_count,
    AVG(salary_amount) AS avg_salary,
    MIN(salary_amount) AS min_salary,
    MAX(salary_amount) AS max_salary
FROM salary;
```

5. Table-Valued Function Equivalent

A table-valued function returns multiple rows. The equivalent using a SELECT query is:

```
SELECT
    user_name,
    department
FROM users
WHERE department = 'IT';
```

Or with a join:

```
SELECT
    u.user_name,
    u.department,
    s.salary_amount
FROM users u
JOIN salary s
    ON u.user_id = s.user_id
WHERE u.department = 'IT';
```

Quick Understanding

Concept	Simple SELECT Example	
-----	-----	
Scalar	`salary_amount * 12`	
Aggregate	`SUM(salary_amount)`	
Table-Valued	`SELECT * FROM users WHERE department='IT'`	

For a workshop, most students understand the concepts faster if you first show these `SELECT` queries and then explain that PostgreSQL allows you to wrap the same logic inside reusable functions.

Concept	Purpose
Non-Recursive CTE	Simplify complex queries
Recursive CTE	Hierarchical data processing
Scalar Function	Returns one value
Aggregate Function	Returns one summarized value
Table-Valued Function	Returns multiple rows
CASE	IF-ELSE logic in SQL
COALESCE	Replace NULL values
NULLIF	Convert matching values to NULL

A **scalar function** accepts one or more input parameters and returns a single value. It does not return multiple rows or multiple columns.

Examples:

UPPER()
 LOWER()
 LENGTH()
 CONCAT()
 ROUND()
 ABS()

Network/Graph, Dimensional/Star

Network, Graph, Dimensional, and Star are **data modeling (database design) concepts**.

They answer the question:

"How should we organize and relate data before writing queries?"

Examples

- **Network Model** → Design for many-to-many relationships.
- **Graph Model** → Design for highly connected data (friends, fraud networks, recommendations).
- **Dimensional Model** → Design for reporting and analytics.
- **Star Schema** → A specific dimensional design with one fact table and multiple dimension tables.

1. Network Model — University Story

Imagine a college.

- * One student can enroll in many courses.
- * One course can have many students.

user1 ---> Java
 ---> Python

user2 ---> Java

---> SQL

This is a many-to-many network of relationships.

One-liner: Network model is used when entities have many-to-many relationships.

2. Graph Model —insta

User1 join insta.

User1 ---- Friend ---- User2

|

Follows

|

David

insta wants to answer:

- * Mutual friends?
- * Friend recommendations?
- * Friends of friends?

This is naturally a graph.

One-liner: Graph model is used to analyze connections between people, accounts, devices, or transactions.

3. Dimensional Model — Amazon Reporting Story

Amazon CEO asks:

> "Show me total sales by product, customer, and month."

Operational tables are complex.

So data is organized as:

Sales Fact
Customer Dimension
Product Dimension
Date Dimension

Optimized for reporting.

One-liner: Dimensional model is designed for analytics and reporting.

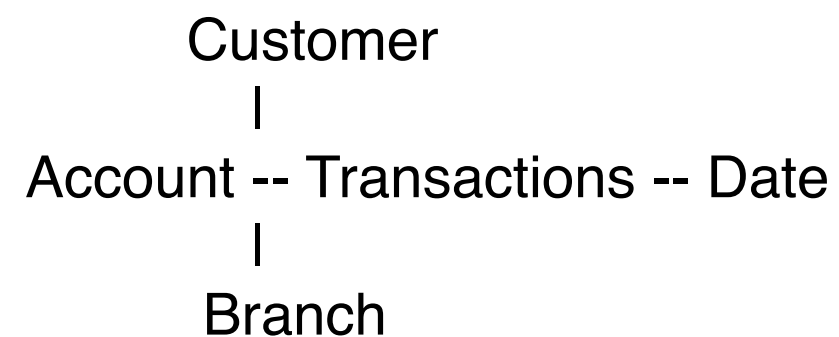
4. Star Schema — BFSI Bank Story

A bank wants Power BI reports.

Center table: Fact_Transactions

- Surrounded by:
- Dim_Customer
- Dim_Account
- Dim_Branch
- Dim_Date

Looks like a star:



Power BI can easily answer:

- * Total transactions by branch
- * Top customers
- * Monthly revenue

One-liner: Star schema is a dimensional model where one fact table is surrounded by dimension tables.

Non-Correlated Subquery

A subquery that can execute independently of the outer query.

```
SELECT *  
FROM salary  
WHERE salary_amount >  
(  
    SELECT AVG(salary_amount)  
    FROM salary  
);
```

Here:

```
SELECT AVG(salary_amount)  
FROM salary
```

runs once and returns a value.

The outer query then uses that value.

Correlated Subquery

A subquery that depends on the outer query.

```
SELECT
    u.user_name,
    u.department
FROM users u
WHERE EXISTS (
    SELECT 1
    FROM salary s
    WHERE s.user_id = u.user_id
);
```

Notice:

`s.user_id = u.user_id`

The subquery references `u.user_id` from the outer query.

Therefore, the subquery executes for each row of the outer query.

Views

Why Use Views?

Simplify complex joins

Reuse common queries

Improve security

Present data in a business-friendly format

```
CREATE VIEW "sample_schema".employee_salary_view AS
SELECT
u.user_id,
u.user_name,
s.salary_amount
FROM users u
JOIN salary s
ON u.user_id = s.user_id;
```

Real-Time Story

Without a view:

```
SELECT
    u.user_name,
    s.salary_amount
FROM users u
JOIN salary s
    ON u.user_id = s.user_id;
```

Developers must write this join repeatedly.

With a view:

```
SELECT * FROM employee_salary_view;
```

The complex query is hidden.